

Requirements Specification: Full-Stack Library Project

1. Project Objective

Build a "Smart Library" application that fetches data from an external API and persists it in a local database. The primary objective is to demonstrate mastery of **3rd Normal Form (3NF)** by designing a relational schema that eliminates all redundant data and transitive dependencies and frontend basics knowledge.

2. Mandatory Technical Stack

- **Frontend:** Vanilla HTML5, CSS3 (Flexbox/Grid), and JavaScript (ES6+).
 - **Backend:** Python 3.x (Flask or FastAPI).
 - **Database:** SQLite3 using the `sqlite3` module.
 - **Prohibited:** No AI tools (ChatGPT, etc.), no frameworks (React, etc.), no CSS libraries (Bootstrap, etc.), and **no ORMs** (Raw SQL only).
-

3. The Database Challenge

The learner must design a database schema based on the following business rules. The design will be evaluated on its adherence to **3rd Normal Form**.

Data Requirements:

When a user saves a book, the following pieces of information must be stored:

1. **Book Details:** Title and ISBN.
2. **Author Details:** Full Name.
3. **Publisher Details:** Company Name and the City where their headquarters is located.
4. **User Tracking:** The date the book was added and the user's current status (e.g., "Want to Read", "Reading", "Finished").

Normalization Constraints:

- The schema must eliminate **Transitive Dependencies** (where a non-key attribute depends on another non-key attribute).
- Data must be **Atomic** (no multiple values in a single cell).
- The learner must be able to justify why their design qualifies as 3NF during the code review.

4. Functional Requirements

Part A: External Data Acquisition

- **API Integration:** Use the [Open Library Search API](#).
- **Data Extraction:** The JavaScript must extract the title, author, publisher, and publisher's location from the API's JSON response to send to the backend.

Part B: The Python Backend

- **RESTful Routes:** * `POST /api/books`: Accept JSON data and perform the necessary logic to insert data into the normalized tables without creating duplicate entries for authors, publishers, or cities.
 - `GET /api/books`: Return a complete list of books.
 - `PATCH /api/books/<id>`: Update the reading status.
- **Relational Integrity:** Ensure foreign key constraints are enabled in SQLite.

Part C: The Frontend UI

- **Search Interface:** A functional search bar that renders results dynamically.
- **Library Dashboard:** A view that displays all saved books. To display the publisher's city and the author's name, the learner must use **SQL JOINS** in the backend.

5. Learning Deliverables

1. **Entity-Relationship Diagram (ERD):** A visual map of the database tables and their relationships created *before* coding the backend.
2. **The Application:** A fully functional project where searching a book and clicking "Save" correctly populates all normalized tables in the database.

6. Evaluation Criteria

- **Database:** Does the design resolve the dependency between the Publisher and the City?
- **SQL Proficiency:** Are JOINS used correctly to fetch data, or is the learner "cheating" by storing duplicate data?

- **Code Quality:** Is the JavaScript clean and free of **innerHTML** for data-heavy tasks?
- **Integrity:** Is there any evidence of AI-generated code? (Manual coding only).